How the Tsuboyama “thermodynamic model” works

Overview

Based strictly on the implementation in external/tsuboyama/additive_model.py, the so-called thermodynamic model is a Bayesian two-way additive model fitted independently for every pair of mutated positions.

For a double mutant containing amino acid a at the first position and amino acid b at the second position, it assumes:

$$dG_{ab} = f_a + s_b + \varepsilon_{ab}$$

- f_a is the fitted effect of amino acid a at the first position.
- s_b is the fitted effect of amino acid b at the second position.
- epsilon_ab is the remaining non-additive residual.

The residual is saved as thermodynamic_coupling and used as experimental epistasis.

1. Only double mutants are modeled

```
mask = out['num_mutations'] == 2
doubles = out.loc[mask].copy()
```

Single mutants, wild type, and variants containing more than two mutations are not used. Consequently, the additive effects are not estimated from measured single-mutant effects.

2. Fits are independent for every position pair

```
for pname, sub in doubles.groupby('pair_name'):
```

A pair_name identifies two mutated sequence positions. All observed amino-acid combinations at those two positions are fitted together. This is often described as a 20 by 20 matrix, but the code does not require all 400 combinations to be observed; it fits the available cells.

3. Mutant amino acids become integer indices

```
aas = 'QENHDRKTSAGMCLVIWYFP'
doubles['aa1'] = [aas.index(m.split(':')[0][-1]) for m in doubles['mutant']]
doubles['aa2'] = [aas.index(m.split(':')[1][-1]) for m in doubles['mutant']]
```

For a mutant such as A10V:G20L, the model extracts V and L. The wild-type amino acids and position numbers do not enter the statistical equation directly; pair_name determines which two positions are being fitted.

4. Forty additive amino-acid effects are sampled

```
first = numpyro.sample('first_aa_effect', dist.Normal(np.resize(0.1, (20)), 3))
second = numpyro.sample('second_aa_effect', dist.Normal(np.resize(0.1, (20)), 3))
```

For every position pair, the model contains 20 effects for the first position and 20 effects for the second position:

$$f_a \sim \mathcal{N}(0.1, 3^2), \qquad s_b \sim \mathcal{N}(0.1, 3^2)$$

The priors are broad. There is no separate intercept; the expected score is represented by the sum of the two amino-acid effects.

5. The additive prediction is constructed

```
pred = [first[i] for i in first_idx] + [second[j] for j in second_idx]
```

For combination ab:

$$\mu_{ab} = f_a + s_b$$

The contribution of amino acid a at the first position is assumed to be the same regardless of the amino acid at the second position, and vice versa. Combination-specific behavior cannot be represented by this additive term.

6. Experimental predictions are clipped in the likelihood

```
if clip_predictions:
    pred = jax.numpy.clip(pred, -1, 5)
```

With the default experimental fit:

$$\widetilde{\mu}_{ab} = \text{clip}(f_a + s_b, -1, 5)$$

Sums below -1 are treated as -1 and sums above 5 as 5. In our PLM-score fits, including conditional MM and PLL, we use clip_predictions=False because model scores do not use the experimental dG scale.

7. Gaussian observation noise is assumed

```
sigma = numpyro.sample('sigma', dist.Exponential(1))
numpyro.sample('obs_dg', Normal(pred, sigma), obs=obs)
```

The actual code expresses the normal distribution as an affine transformation of a standard normal, but it is equivalent to:

$$\sigma \sim \text{Exponential}(1)$$

$$dG_{ab} \sim \mathcal{N}(\widetilde{\mu}_{ab}, \sigma^2)$$

All double mutants in one position-pair fit share the same noise parameter.

8. NUTS samples the posterior

```
kernel = NUTS(additive_model, init_strategy=initialization.init_to_feasible())
mcmc = MCMC(kernel, num_warmup=100, num_samples=50, num_chains=1)
```

Each position pair is fitted with one MCMC chain, 100 warm-up iterations, and 50 retained posterior samples. NUTS jointly samples 20 first-position effects, 20 second-position effects, and one noise parameter. The code records divergences, mean acceptance probability, and maximum NUTS steps as diagnostics.

9. Posterior samples become a single additive reconstruction

```
first = np.median(samples['first_aa_effect'], axis=0)
second = np.median(samples['second_aa_effect'], axis=0)
recon = [first[a] + second[b] for a, b in zip(sub['aa1'], sub['aa2'])]
```

The code takes the marginal posterior median for each amino-acid effect and then adds them:

$$\widehat{dG}^{add}_{ab} = \text{median}(f_a) + \text{median}(s_b)$$

This value is saved as recon_dg.

10. Epistasis is the residual

```
thermodynamic_coupling = dG - recon_dg
```

Therefore:

$$\varepsilon_{ab} = dG_{ab} - \widehat{dG}^{add}_{ab}$$

- A residual near zero means the score is explained by the additive reconstruction.
- A positive residual means the observed score is above the additive reconstruction.
- A negative residual means it is below the additive reconstruction.
- A large absolute residual means a strong departure from additivity.

The precise biochemical meaning of the sign depends on the direction of the score. The safest interpretation is “above or below the fitted additive expectation on this score scale.”

Concrete example

Suppose the fit estimates:

$$\hat{f}_V = 0.8, \qquad \hat{s}_L = 1.1$$

The additive reconstruction for combination VL is 1.9. If the measured score is 2.5:

$$\varepsilon_{VL} = 2.5 - 1.9 = 0.6$$

The combination lies 0.6 units above its fitted additive expectation.

Important implementation subtleties

The saved reconstruction is not clipped

Clipping is applied to pred inside the likelihood during fitting. After fitting, however, recon_dg is calculated as first[a] + second[b] without clipping. Thus, when clip_predictions=True, posterior fitting uses a clipped mean but the saved reconstruction and residual use the un-clipped sum.

It sums marginal posterior medians

The implementation calculates:

$$\text{median}(f_a) + \text{median}(s_b)$$

It does not calculate:

$$\text{median}(f_a + s_b)$$

These are often similar but are not mathematically guaranteed to be identical.

The individual effects are not uniquely identifiable

For any constant c:

$$(f_a + c) + (s_b - c) = f_a + s_b$$

Therefore, individual first- and second-position coefficients should not be interpreted as uniquely determined biochemical effects. The priors partially anchor the decomposition. The fitted sums and residuals are the relevant outputs.

The random seed is reset for every pair

The code initializes PRNGKey(1) inside the loop. Every position-pair fit therefore uses the same deterministic random-number stream. This makes runs reproducible but does not give independent random seeds across pairs.

Posterior sampling is short

Only 50 posterior samples from one chain are retained after 100 warm-up iterations. This reproduces the established workflow, but individual pair-level posterior estimates should be interpreted with the MCMC diagnostics in mind.

Why it is called “thermodynamic”

The model does not simulate folding equilibria, residue contacts, molecular energies, or a mechanistic thermodynamic cycle. It is called thermodynamic because it is applied to experimental stability-related dG values.

Statistically, it is a Bayesian two-factor additive model with Gaussian noise:

```
observed double-mutant score
    = fitted additive component
    + residual coupling
```

For a protein-language-model score, we repeat the same decomposition:

$$S_{ab} = \widehat{S}_{ab}^{add} + \varepsilon_{ab}^{pred}$$

Predicted epistasis is then compared with experimental thermodynamic coupling using Spearman correlation.